

net-06 - Introdução ao OVN

Contexto

Neste treinamento iremos abordar:

- Uma breve introdução ao OVN
- Arquitetura do OVN
- Fluxo de informações entre componentes do OVN
- Explorando o Chassi
- OVN Central em cluster
- Logical Flows
- Hands on - Instalação do OVN central/OVN-controller e OVS
- Hands on - Explorando os Logical Flows
- Hands on - Criando topologias de rede avançadas
- Hands on - Fazendo um paralelo entre Logical Flows, Openflow e Physical Flows
- Hands on - Explorando os logs

Introdução ao OVN

Neste treinamento iremos introduzir o **OVN** (Open Virtual Network), que é um projeto de código aberto que foi originalmente desenvolvido pela comunidade do Open vSwitch (OVS).

Ao longo dos treinamentos anteriores, foi mencionado algumas vezes o funcionamento do modelo de SDN. Pois bem, o OVN é uma solução de SDN que complementa os recursos existentes do OVS e fornece abstração de rede virtual (L2 e L3), security group, serviços DHCP e dentre outros.

Um deployment de OVN consiste em muitos componentes:

- Um Cloud Management System (CMS) que é uma aplicação externa. Geralmente podemos posicionar o Openstack como um CMS mas existem outros clientes como K8S e LXD
- Uma database central que possui redundância lógica por funcionar em Cluster
- Um ou muitos hypervisors(compute nodes) que devem ter o OVS e que são identificados como Chassis
- Nenhum ou vários Chassi Gateway que são responsáveis em criar conectividade entre redes virtuais do Overlay com redes externas no Underlay (ex: Internet). Também é conhecido como Chassi Gateway.

Arquitetura do OVN

Basicamente o OVN se divide em duas camadas:

- **OVN Central** - Que é a SDN Controller
- **Chassi com ovn-controller** - Hypervisors/Gateway Nodes que executam o OVS e que é gerenciado pela SDN Controller

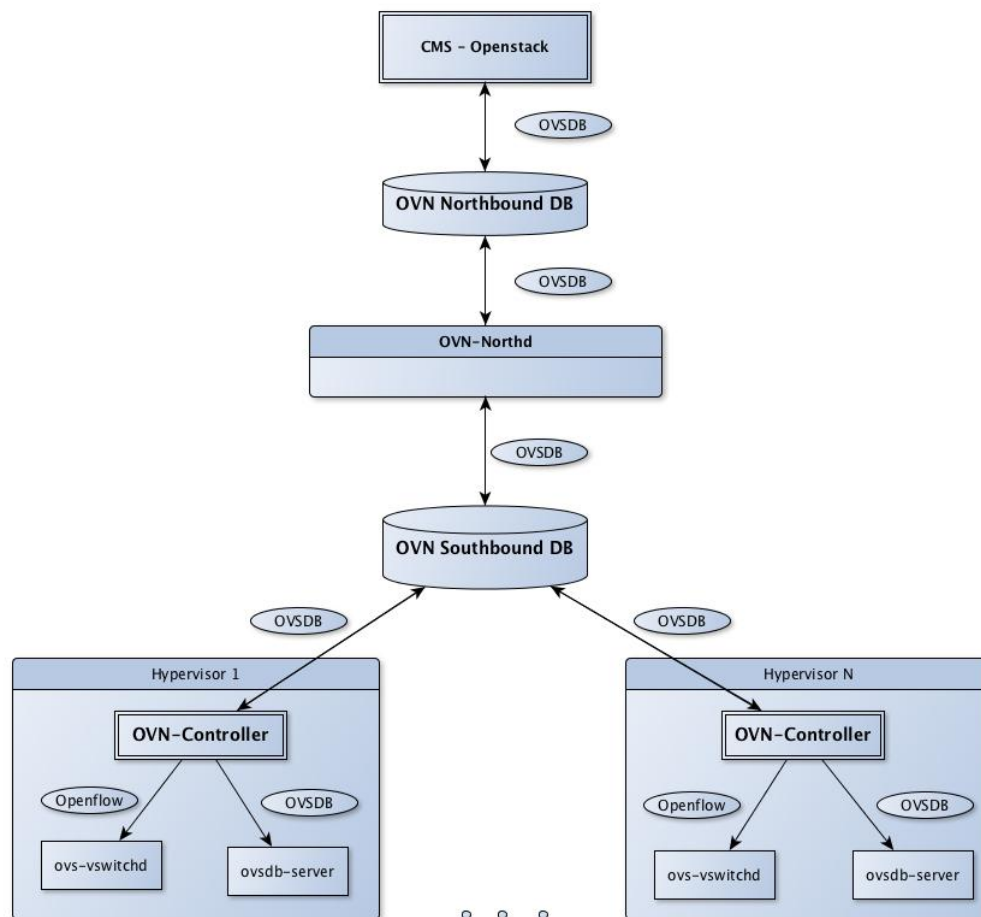
O diagrama abaixo mostra os principais componentes do OVN e a interação entre eles:

- **CMS** - conforme já mencionado anteriormente sobre o Openstack, temos o Neutron com um plugin(ML2/OVN) de integração com o OVN. O principal propósito do plugin é traduzir toda a configuração lógica de rede armazenada no banco de dados do Neutron em um formato intermediário entendido pelo OVN
- **OVN Northbound database** - recebe o formato intermediário das configurações lógicas de rede enviadas pelo plugin do Neutron(ML2/OVN) em uma database. O schema da database deve usar conceitos utilizados do lado CMS, de modo que ele suporte diretamente noções de logical switch, routers, ACLs e assim por diante. O OVN Northbound database possui dois clientes, o CMS acima dele e o ovn-northd.
- **ovn-northd** - se conecta com o OVN Northbound database e com o OVN Southbound database. É responsável em traduzir a configuração lógica de rede a partir do OVN Northbound para logical datapath flows e inserir no OVN Southbound database.

- **OVN Southbound database** - é a parte central da solução, seus clientes são o `ovn-northd` e o `ovn-controller` que está presente em todos os hypervisors(Chassis)/Gateway Nodes. O OVN Southbound database possui 3 tipos de dados: Physical Network (PN) tables - especificam como alcançar os hypervisors, Logical Network (LN) tables - descreve a configuração lógica de rede em logical datapath flows e Binding tables - faz uma ligação entre a localização da configuração lógica de rede com a Physical Network.
 - Os hypervisors/Gateway Nodes populam as Physical Network tables e Port_Binding enquanto o `ovn-northd` popula as Logical Network tables

Os componentes remanescentes são replicados em cada hypervisor/Gateway Node:

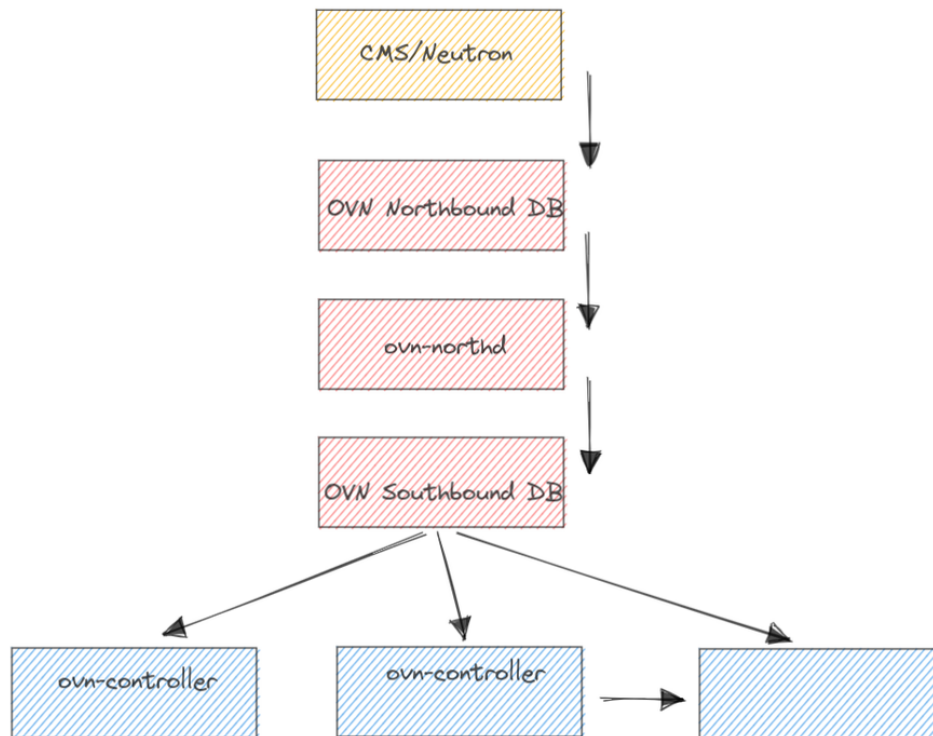
- **ovn-controller** - é um agente que executa em cada hypervisor/Gateway Node. Para o norte, ele se conecta no OVN Southbound database para aprender sobre novas configurações e status, popular a Physical Network table e a coluna **Chassis** na **Binding** table com status do hypervisor/Gateway Node. Para o sul, se conecta com o **ovs-vswitchd** como uma controller Openflow e com o **ovsdb-server** para permitir o monitoramento e controle da configuração do OVS.



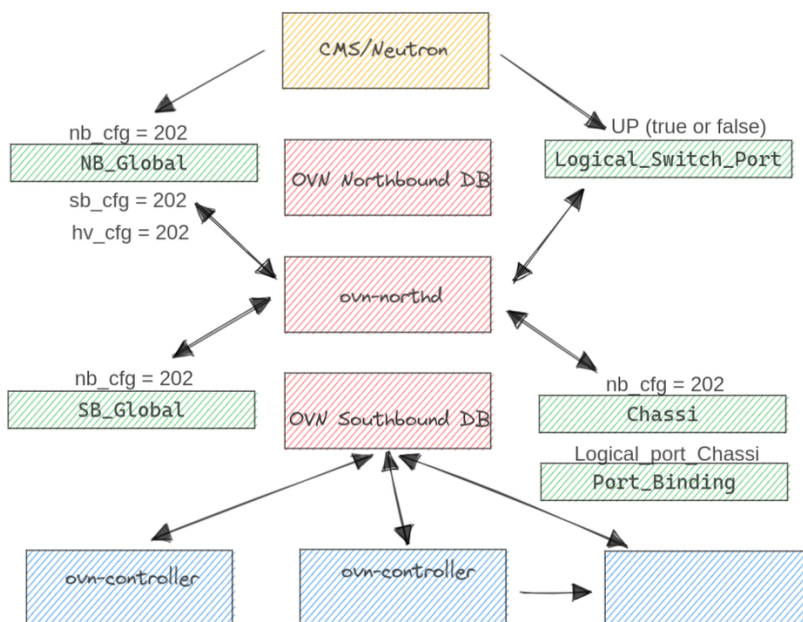
Fluxo de informações entre componentes do OVN [↗](#)

Os fluxos principais de informações são de configuração e de status no OVN:

- O fluxo de configuração de qualquer objeto no OVN sempre é do North para South. O CMS através de seu plugin do OVN irá enviar a configuração lógica de rede para o **ovn-northd** via **OVN Northbound database**. Por sua vez, o **ovn-northd** irá compilar essa configuração para um formato de nível inferior e irá enviar para todos os hypervisors/Gateway Nodes via **OVN Southbound database**.



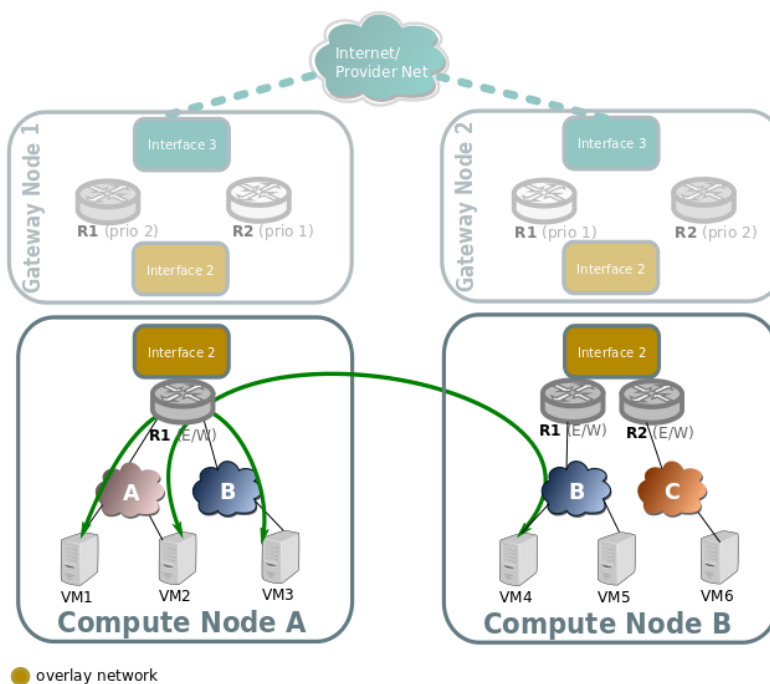
- O fluxo de status sempre é do South para North. O OVN atualmente provê algumas formas de informação de status:
 - Primeiro, para que o CMS detecte quando uma VM está em execução, o **ovn-northd** irá monitorar o valor da column **logical port Chassi** (true ou false) da **OVN Southbound database Port_Binding** e atualizar a column **UP** na **OVN Northbound database table Logical_Switch_Port**.
 - Segundo, o OVN provê feedback para o CMS quando uma configuração solicitada pelo CMS entra em vigor. Nesse fluxo, existe uma sequência de passos:
 - Quando o CMS solicita configurações para o **OVN Northbound database**, ao mesmo tempo é incrementado o valor da column **nb_cfg** na **NB_Global** table
 - Quando o **ovn-northd** compila as informações a partir do **OVN Northbound database** e inseri no **OVN Southbound database**, o mesmo copia o valor da column **nb_cfg** do North para a table **SB_Global** do **OVN Southbound database**
 - Após o **ovn-northd** receber a confirmação que a configuração foi aplicada, ele irá atualizar a column **sb_cfg** na table **NB_Global** do **OVN Northbound database** para o valor de **nb_cfg** que foi incrementado anteriormente
 - O **ovn-controller** irá processar em cada hypervisor as modificações enviadas via **OVN Southbound database** de acordo com o valor da column **nb_cfg** da table **SB_Global**. Após os physical flows serem instalados no OVS, o **ovn-controller** irá atualizar o valor de **nb_cfg** de seu respectivo hypervisor na table **Chassi** do **OVN Southbound database**
 - O **ovn-northd** monitora a column **nb_cfg** de todos os Chassis da table **Chassi** no **OVN Southbound database** e copia os valores atualizados para a column **hv_cfg** da table **NB_Global** do **OVN Northbound database**. Dessa forma o CMS pode saber que todas as configurações solicitadas foram aplicadas em todos os hypervisors.



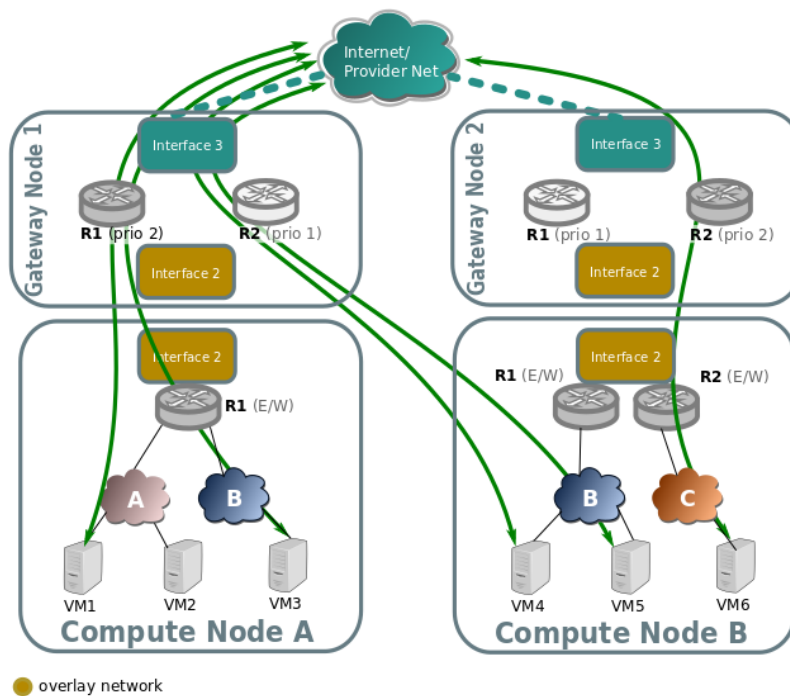
Explorando o Chassi

Um Chassi pode ser classificado em dois tipos 1) **Chassi(hypervisor)** e 2) **Chassi Gateway**, ambos rodam os mesmos componentes do OVN (**ovn-controller**) e OVS porém executam funções diferentes.

- **Chassi** - É onde as VMs estarão em execução e todo o tráfego de rede **leste-oeste** é gerenciado por esse tipo de Chassi



- **Chassi Gateway** - Responsável em criar conectividade **norte-sul** com ambiente externos (Ex: Internet)



O **Chassi Gateway** é configurado modificando o valor do **ovn-cms-options** para **enable-chassis-as-gw** localmente no OVS.

Em cada **Chassi** será configurado o OVS com uma bridge dedicada para o OVN, chamada bridge integration (br-int). A bridge **br-int** é criada automaticamente durante a inicialização do serviço do **ovn-controller**.

As portas que serão conectadas na bridge **br-int** são:

- Tunnel ports responsáveis em criar conectividade entre Chassis, sendo o ovn-controller responsável em adicionar, atualizar e remover as tunnel ports
- Em um hypervisor, o Nova (componente do Openstack) irá conectar as VIFs das VMs na br-int
- Em um Chassi Gateway, portas físicas utilizadas para criar conectividade norte-sul são adicionadas na br-int

OVN Central em cluster [↗](#)

O OVN Central é considerado como a Control Plane ou SDN Controller da solução do OVN e é composto pelos seguintes serviços:

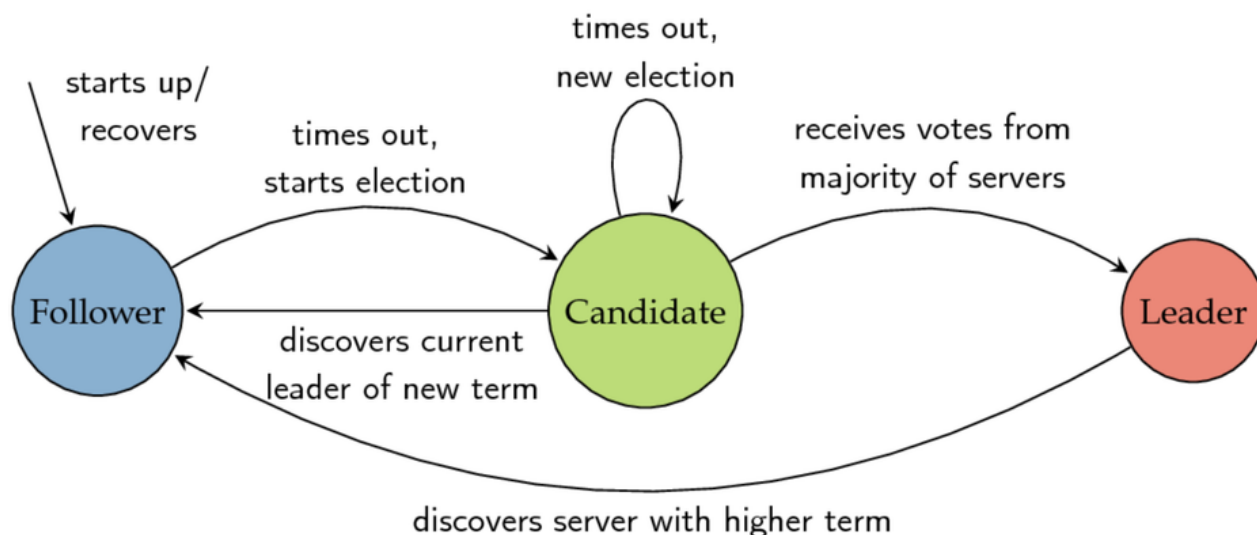
- **OVN Northbound database**
- **ovn-northd**
- **OVN Southbound database**

As databases de Northbound e Southbound do OVN são gerenciadas pelo serviço do **ovsdb-server** que possui alguns modelos de serviço para prover disponibilidade das databases:

- **Standalone** - executa em um único servidor e caso o mesmo tenha algum problema as databases ficarão indisponíveis para o restante dos componentes da solução
- **Active-Backup** - executa em 2 ou 3 servidores que estão em lugares distintos para prover alta disponibilidade. Um dos servidores atua como **Active** para responder as requisições dos componentes da solução de acesso as databases. O(s) outro(s) servidor(ers) atua(m) como **Backup** e faz a replicação das databases a partir do servidor Active. Nesse modelo de serviço é utilizado a solução de alta disponibilidade Pacemaker para gerir o cluster que é responsável em ativar os serviços do OVN Central no servidor Active, configurar recursos como Virtual IPs utilizados pelos clientes das databases e gerenciar o failover do Active para o Backup quando necessário
- **Clustered** - executa em 3 ou 5 servidores que estão em lugares distintos para prover alta disponibilidade. Os servidores em cluster sincronizam a escrita entre eles automaticamente. Um cluster com 3 servidores pode tolerar a falha de 1 servidor e um cluster com 5 servidores pode suportar a falha de 2.

No modelo de serviço com cluster, utiliza-se o **Raft** para gerenciar o cluster que permitirá a escrita e leitura das databases conforme o status dos membros do cluster. Em um cluster, ocorrerá a eleição do **Leader** que permitirá a escrita/leitura em suas databases locais enquanto os **Followers** sincronizam as alterações a partir do Leader e permitem apenas a leitura local das databases. Em uma eleição os seguintes status são esperados:

- **Follower** - Esse é o primeiro estado após o servidor entrar no cluster
- **Candidate** - Quando não existe um Leader no cluster, se inicia a eleição e o estado muda para Candidate
- **Leader** - Se receber a maioria dos votos, o estado passa para Leader e o restante dos servidores mudam seu estado para Follower



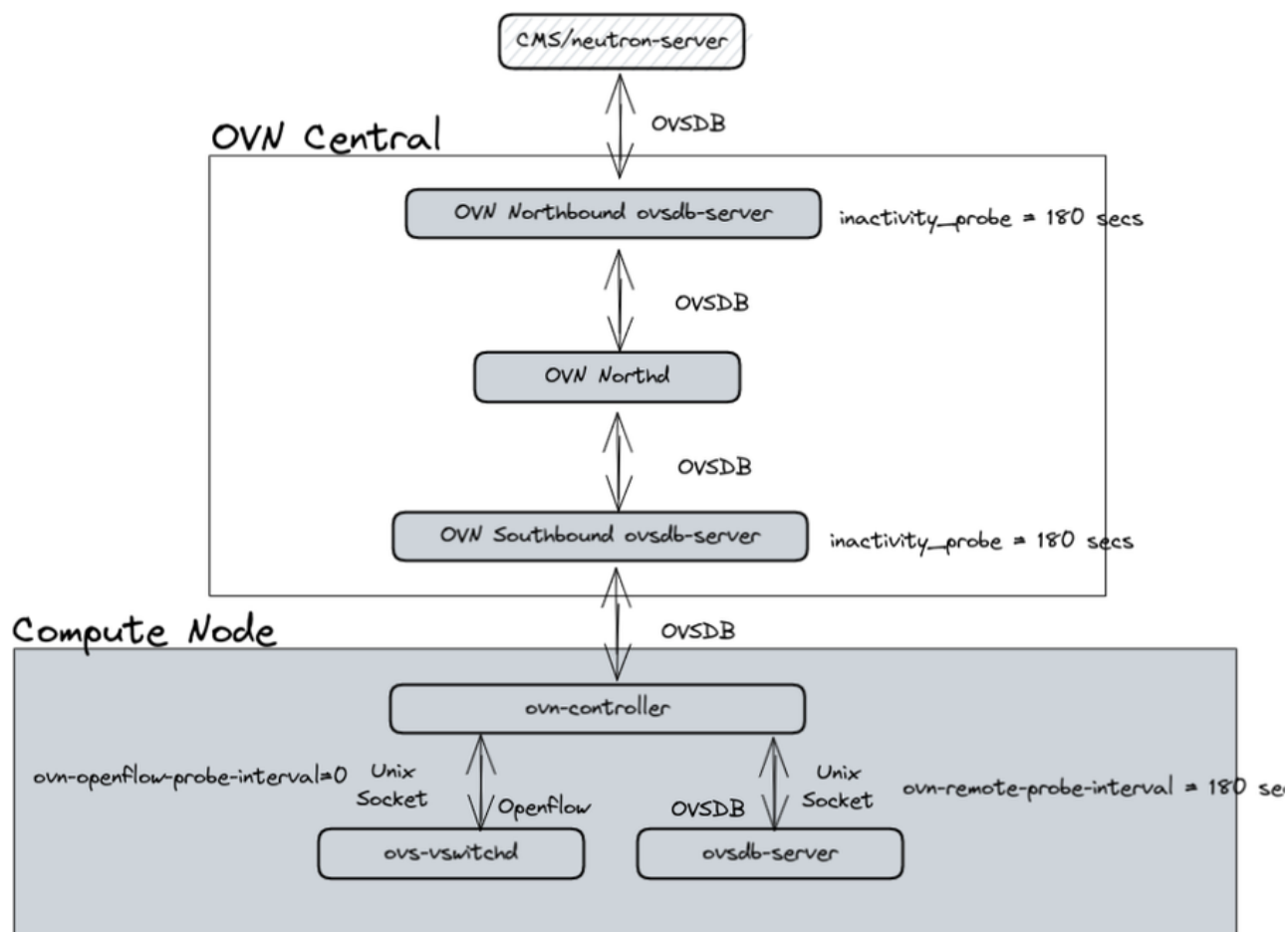
i O Leader é definido para o Northbound e também para o Southbound

Para evitar que eleições ocorram inadvertidamente, os servidores do cluster possuem um timer de 10 secs para monitorar o Leader. No caso do Leader ter um problema, após os 10 segundos é realizada uma reeleição.

Uma reeleição do Leader também ocorre quando o **ovsdb-server** deve executar a rotina de snapshot das databases que não é configurável e ocorre conforme a massa de dados das databases.

Para que os componentes do OVN Central possam processar as requisições dos clientes e evitar gargalos, existem timers que são configurados entre os componente da solução:

- OVN Central - **inactivity_probe** (Northbound/Southbound) = **180 secs**
- OVN-controller - **ovn-remote-probe-interval** = **180 secs** e **ovn-openflow-probe-interval** = **0 secs**



As portas TCP de serviço utilizadas pelos vários daemons do OVN Central são:

- 6641 - OVN Northbound database
- 6642 - OVN Southbound database
- 6643 - Utilizado pelo Raft para o Northbound
- 6644 - Utilizado pelo Raft para o Southbound

Logical Flows [↗](#)

Nos últimos treinamentos, foi detalhado o funcionamento do OVS e como programar os pipelines de processamento via Openflow manualmente. Programar os pipelines de processamento de pacotes em poucos hosts pode ser uma tarefa desafiadora e muito mais quando se trata de dezenas e até centenas de hosts.

O OVN como uma solução de SDN é responsável em programar via Openflow o OVS e de implementar topologias de rede e ACLs utilizando uma interface de alto nível.

Como o OVN determina quais flows são requeridos para cada host? A resposta para isso está nos Logical Flows.

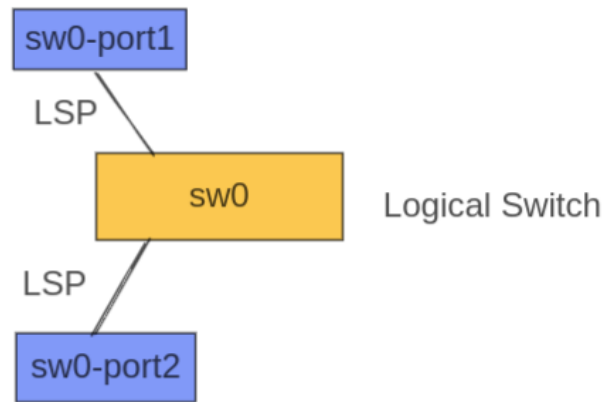
Logical Flows são conceitualmente similar com o **Openflow**, pois são compostos por flow tables com priority, match e actions. E a grande diferença é que os Logical Flows descrevem o comportamento detalhado de uma topologia de rede que pode abranger um grande número de hosts. Isso acaba fornecendo a separação entre definir o comportamento detalhado da rede e ter que se preocupar com o layout físico real da rede.

Portanto, o OVN centralmente programa as topologias de rede em Logical Flows, que são distribuídos através do **ovn-controller** que executam em cada host. Com isso, o **ovn-controller** sabe como traduzir Logical Flows em Openflow usando o estado atual da rede física (que portas residem localmente, como alcançar outros hosts).

Em detalhes mais práticos, Logical Flows podem ser definidos como:

- **Logical Switch** (LS) - Semelhante a um switch ou para uma subnet L2
- **Logical Router** (LR) - Similar a um router (o mesmo é distribuído)
- **Logical Switch Port** (LSP) - Pontos de conectividade de ingress e egress e geralmente são conectadas com VMs
- **Logical Router Port** (LRP) - Pontos de conectividade de ingress e egress entre um Logical Switch e um Logical Router

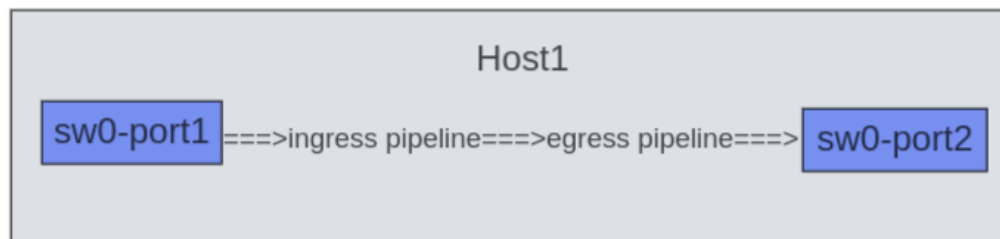
Abaixo vamos exemplificar o funcionamento dos Logical Flows com um Logical Switch:



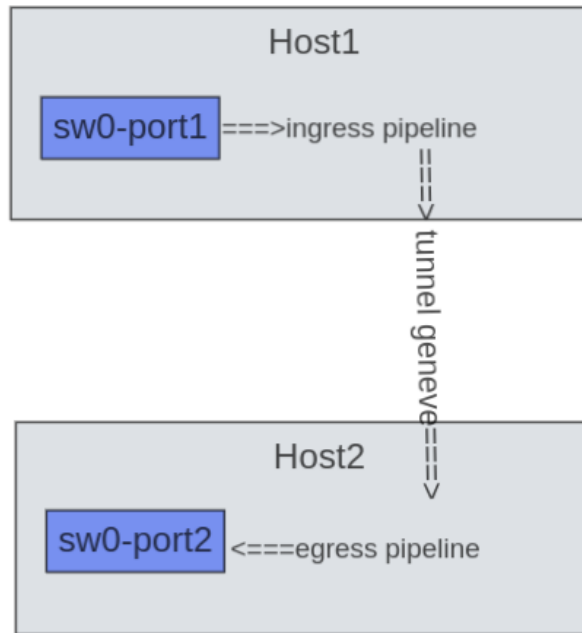
O OVN irá definir o Logical Switch sw0 usando dois pipelines:

- ingress pipeline
- egress pipeline

Quando um pacote entra no Logical Switch **sw0**, o ingress pipeline será executado onde o pacote foi originado. Se o destino é no mesmo host, então o egress pipeline é executado.



Se o destino é remoto (outro host), o pacote será enviado via tunnel antes de executar o egress pipeline:



Instalação do OVN central/OVN-controller e OVS [↗](#)

i Para fins de teste **apenas**, se recomenda ter uma VM(sandbox) executando o Ubuntu 20.04 com os pacotes latest instalados para seguir os exercícios.

1 - Instalando o OVN Central/OVN-controller e OVS via pacote [↗](#)

Neste exercício, o setup irá ser All-in-One e usaremos as últimas versões dos pacotes:

```
1 # add-apt-repository -y cloud-archive:yoga
```

Realizar a instalação dos pacotes necessários:

```
1 # apt update
2 # apt install -y ovn-central ovn-host openvswitch-switch
```

Inicialize os serviços:

```
1 # systemctl start openvswitch-switch
2 # systemctl start ovn-central
```

2 - Checando os serviços OVN Central/OVN-controller e OVS [↗](#)

Após a instalação serão inicializados automaticamente os seguintes serviços:

OVN Central:

- OVN Northbound database
- ovn-northd
- OVN Southbound database

```
1 # systemctl status ovn-central
2 • ovn-central.service - Open Virtual Network central components
3   Loaded: loaded (/lib/systemd/system/ovn-central.service; enabled; vendor preset: enabled)
4   Active: active (exited) since Mon 2022-06-06 21:06:17 UTC; 1min 8s ago
5   Process: 3115 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
6   Main PID: 3115 (code=exited, status=0/SUCCESS)
```

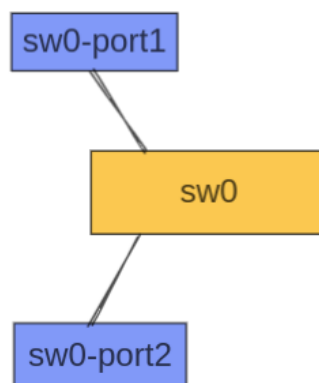
```
7
8 Jun 06 21:06:16 ovn-sandbox1 systemd[1]: Starting Open Virtual Network central components...
9 Jun 06 21:06:17 ovn-sandbox1 systemd[1]: Finished Open Virtual Network central components.
```

OVN-controller e OVS:

```
1 # systemctl status ovn-controller
2 • ovn-controller.service - Open Virtual Network host control daemon
3   Loaded: loaded (/lib/systemd/system/ovn-controller.service; static; vendor preset: enabled)
4   Active: active (running) since Mon 2022-06-06 20:58:50 UTC; 11min ago
5   Main PID: 2133 (ovn-controller)
6   Tasks: 4 (limit: 4725)
7   Memory: 3.0M
8   CGroup: /system.slice/ovn-controller.service
9           └─2133 ovn-controller unix:/var/run/openvswitch/db.sock -vconsole:emer -vsyslog:err -vfile:info --n
10 # systemctl status openvswitch-switch
11 • openvswitch-switch.service - Open vSwitch
12   Loaded: loaded (/lib/systemd/system/openvswitch-switch.service; enabled; vendor preset: enabled)
13   Active: active (exited) since Mon 2022-06-06 21:02:35 UTC; 8min ago
14   Process: 2742 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
15   Main PID: 2742 (code=exited, status=0/SUCCESS)
```

Explorando os Logical Flows - cenário 1 [↗](#)

Neste exercício, iremos explorar os Logical Flows da seguinte topologia de rede com um único domínio L2:



Vamos realizar a criação de um **Logical Switch** com duas **Logical ports** conectadas nesse **LS**:

```
1 # ovn-nbctl ls-add sw0
2
3 # ovn-nbctl lsp-add sw0 sw0-port1
4 # ovn-nbctl lsp-set-addresses sw0-port1 00:00:00:00:00:01
5 # ovn-nbctl lsp-set-port-security sw0-port1 00:00:00:00:00:01
6
7 # ovn-nbctl lsp-add sw0 sw0-port2
8 # ovn-nbctl lsp-set-addresses sw0-port2 00:00:00:00:00:02
9 # ovn-nbctl lsp-set-port-security sw0-port2 00:00:00:00:00:02
```

Explorando o Logical Switch criado com o tool **ovn-nbctl**(nb = northbound):

```
1 # ovn-nbctl show sw0
2 switch de8aa9a4-d52f-4866-ba44-844e6c2f278c (sw0)
3   port sw0-port2
```

```

4     addresses: ["00:00:00:00:00:02"]
5     port sw0-port1
6     addresses: ["00:00:00:00:00:01"]

```

Verifique os pipelines criados automaticamente com o tool **ovn-sbctl**(sb - southbound):

```

1 # ovn-sbctl lflow-list
2 Datapath: "sw0" (813d1dd3-ccf2-4518-9c80-e6eca0f3cd0b) Pipeline: ingress
3 table=0 (ls_in_port_sec_l2 ), priority=100 , match=(eth.src[40]), action=(drop;)
4 table=0 (ls_in_port_sec_l2 ), priority=100 , match=(vlan.present), action=(drop;)
5 table=0 (ls_in_port_sec_l2 ), priority=50 , match=(inport == "sw0-port1" && eth.src == {00:00:00:00:00:01})
6 table=0 (ls_in_port_sec_l2 ), priority=50 , match=(inport == "sw0-port2" && eth.src == {00:00:00:00:00:02})
7 table=1 (ls_in_port_sec_ip ), priority=0 , match=(1), action=(next;)
8 table=2 (ls_in_port_sec_nd ), priority=90 , match=(inport == "sw0-port1" && eth.src == 00:00:00:00:00:01 &&
9 table=2 (ls_in_port_sec_nd ), priority=90 , match=(inport == "sw0-port1" && eth.src == 00:00:00:00:00:01 &&
10 table=2 (ls_in_port_sec_nd ), priority=90 , match=(inport == "sw0-port2" && eth.src == 00:00:00:00:00:02 &&
11 table=2 (ls_in_port_sec_nd ), priority=90 , match=(inport == "sw0-port2" && eth.src == 00:00:00:00:00:02 &&
12 table=2 (ls_in_port_sec_nd ), priority=80 , match=(inport == "sw0-port1" && (arp || nd)), action=(drop;)
13 table=2 (ls_in_port_sec_nd ), priority=80 , match=(inport == "sw0-port2" && (arp || nd)), action=(drop;)
14 table=2 (ls_in_port_sec_nd ), priority=0 , match=(1), action=(next;)
15 table=3 (ls_in_lookup_fdb ), priority=0 , match=(1), action=(next;)
16 table=4 (ls_in_put_fdb ), priority=0 , match=(1), action=(next;)
17 table=5 (ls_in_pre_acl ), priority=110 , match=(eth.dst == $svc_monitor_mac), action=(next;)
18 table=5 (ls_in_pre_acl ), priority=0 , match=(1), action=(next;)
19 table=6 (ls_in_pre_lb ), priority=110 , match=(eth.dst == $svc_monitor_mac), action=(next;)
20 table=6 (ls_in_pre_lb ), priority=110 , match=(eth.mcast), action=(next;)
21 table=6 (ls_in_pre_lb ), priority=110 , match=(nd || nd_rs || nd_ra || mldv1 || mldv2), action=(next;)
22 table=6 (ls_in_pre_lb ), priority=0 , match=(1), action=(next;)
23 table=7 (ls_in_pre_stateful ), priority=120 , match=(reg0[2] == 1 && ip4 && sctp), action=(reg1 = ip4.dst; re
24 table=7 (ls_in_pre_stateful ), priority=120 , match=(reg0[2] == 1 && ip4 && tcp), action=(reg1 = ip4.dst; reg
25 table=7 (ls_in_pre_stateful ), priority=120 , match=(reg0[2] == 1 && ip4 && udp), action=(reg1 = ip4.dst; reg
26 table=7 (ls_in_pre_stateful ), priority=120 , match=(reg0[2] == 1 && ip6 && sctp), action=(xxreg1 = ip6.dst;
27 table=7 (ls_in_pre_stateful ), priority=120 , match=(reg0[2] == 1 && ip6 && tcp), action=(xxreg1 = ip6.dst; r
28 table=7 (ls_in_pre_stateful ), priority=120 , match=(reg0[2] == 1 && ip6 && udp), action=(xxreg1 = ip6.dst; r
29 table=7 (ls_in_pre_stateful ), priority=110 , match=(reg0[2] == 1), action=(ct_lb;)
30 table=7 (ls_in_pre_stateful ), priority=100 , match=(reg0[0] == 1), action=(ct_next;)
31 table=7 (ls_in_pre_stateful ), priority=0 , match=(1), action=(next;)
32 table=8 (ls_in_acl_hint ), priority=65535, match=(1), action=(next;)
33 table=9 (ls_in_acl ), priority=65535, match=(1), action=(next;)
34 table=10(ls_in_qos_mark ), priority=0 , match=(1), action=(next;)
35 table=11(ls_in_qos_meter ), priority=0 , match=(1), action=(next;)
36 table=12(ls_in_lb ), priority=0 , match=(1), action=(next;)
37 table=13(ls_in_acl_after_lb ), priority=0 , match=(1), action=(next;)
38 table=14(ls_in_stateful ), priority=100 , match=(reg0[1] == 1 && reg0[13] == 0), action=(ct_commit { ct_1
39 table=14(ls_in_stateful ), priority=100 , match=(reg0[1] == 1 && reg0[13] == 1), action=(ct_commit { ct_1
40 table=14(ls_in_stateful ), priority=0 , match=(1), action=(next;)
41 table=15(ls_in_pre_hairpin ), priority=0 , match=(1), action=(next;)
42 table=16(ls_in_nat_hairpin ), priority=0 , match=(1), action=(next;)
43 table=17(ls_in_hairpin ), priority=0 , match=(1), action=(next;)
44 table=18(ls_in_arp_rsp ), priority=0 , match=(1), action=(next;)
45 table=19(ls_in_dhcp_options ), priority=0 , match=(1), action=(next;)
46 table=20(ls_in_dhcp_response), priority=0 , match=(1), action=(next;)
47 table=21(ls_in_dns_lookup ), priority=0 , match=(1), action=(next;)
48 table=22(ls_in_dns_response ), priority=0 , match=(1), action=(next;)
49 table=23(ls_in_external_port), priority=0 , match=(1), action=(next;)
50 table=24(ls_in_l2_lkup ), priority=110 , match=(eth.dst == $svc_monitor_mac), action=(handle_svc_check(i
51 table=24(ls_in_l2_lkup ), priority=70 , match=(eth.mcast), action=(output = "_MC_flood"; output;))
52 table=24(ls_in_l2_lkup ), priority=50 , match=(eth.dst == 00:00:00:00:00:01), action=(output = "sw0-po
53 table=24(ls_in_l2_lkup ), priority=50 , match=(eth.dst == 00:00:00:00:00:02), action=(output = "sw0-po

```

```

54 table=24(ls_in_l2_lkup      ), priority=0    , match=(1), action=(output = get_fdb(eth.dst); next;)
55 table=25(ls_in_l2_unknown  ), priority=50  , match=(output == "none"), action=(drop;)
56 table=25(ls_in_l2_unknown  ), priority=0    , match=(1), action=(output;)
57 Datapath: "sw0" (813d1dd3-ccf2-4518-9c80-e6eca0f3cd0b) Pipeline: egress
58 table=0 (ls_out_pre_lb     ), priority=110 , match=(eth.mcast), action=(next;)
59 table=0 (ls_out_pre_lb     ), priority=110 , match=(eth.src == $svc_monitor_mac), action=(next;)
60 table=0 (ls_out_pre_lb     ), priority=110 , match=(nd || nd_rs || nd_ra || mldv1 || mldv2), action=(next;)
61 table=0 (ls_out_pre_lb     ), priority=0    , match=(1), action=(next;)
62 table=1 (ls_out_pre_acl    ), priority=110 , match=(eth.src == $svc_monitor_mac), action=(next;)
63 table=1 (ls_out_pre_acl    ), priority=0    , match=(1), action=(next;)
64 table=2 (ls_out_pre_stateful), priority=110 , match=(reg0[2] == 1), action=(ct_lb;)
65 table=2 (ls_out_pre_stateful), priority=100 , match=(reg0[0] == 1), action=(ct_next;)
66 table=2 (ls_out_pre_stateful), priority=0    , match=(1), action=(next;)
67 table=3 (ls_out_acl_hint   ), priority=65535, match=(1), action=(next;)
68 table=4 (ls_out_acl        ), priority=65535, match=(1), action=(next;)
69 table=5 (ls_out_qos_mark   ), priority=0    , match=(1), action=(next;)
70 table=6 (ls_out_qos_meter  ), priority=0    , match=(1), action=(next;)
71 table=7 (ls_out_stateful   ), priority=100 , match=(reg0[1] == 1 && reg0[13] == 0), action=(ct_commit { ct_l
72 table=7 (ls_out_stateful   ), priority=100 , match=(reg0[1] == 1 && reg0[13] == 1), action=(ct_commit { ct_l
73 table=7 (ls_out_stateful   ), priority=0    , match=(1), action=(next;)
74 table=8 (ls_out_port_sec_ip), priority=0    , match=(1), action=(next;)
75 table=9 (ls_out_port_sec_l2), priority=100 , match=(eth.mcast), action=(output;)
76 table=9 (ls_out_port_sec_l2), priority=50  , match=(output == "sw0-port1" && eth.dst == {00:00:00:00:00:01}
77 table=9 (ls_out_port_sec_l2), priority=50  , match=(output == "sw0-port2" && eth.dst == {00:00:00:00:00:02}

```

No output acima, pode-se notar que temos um Datapath para o Logical Switch sw0 para o pipeline de ingress e um segundo Datapath para o pipeline de egress.

Para entender os Logical Flows mais a fundo, podemos utilizar o tool **ovn-trace**. O ovn-trace permitirá que você verifique como o OVN processará os pipelines com um sample packet.

Ele requer no mínimo 2 parâmetros:

```
1 # ovn-trace DATAPATH MICROFLOW
```

DATAPATH - Identifica o Logical Datapath (Logical Switch ou Logical Router) onde o sample packet irá iniciar

MICROFLOW - Descreve o sample packet que irá ser simulado

Vamos verificar como nosso cenário irá processar um sample packet, o ovn-trace suporta vários níveis de detalhamento, vamos explorar um por um e para iniciar será com o nível minimal.

No primeiro teste, vamos simular um envio de um sample packet entre a porta sw0-port1 para a porta sw0-port2:

```

1 # ovn-trace --minimal sw0 'inport == "sw0-port1" && eth.src == 00:00:00:00:00:01 && eth.dst == 00:00:00:00:00:02'
2 # reg14=0x1,vlan_tci=0x0000,d1_src=00:00:00:00:00:01,d1_dst=00:00:00:00:00:02,d1_type=0x0000
3 output("sw0-port2");

```

Acima podemos determinar que o sample packet será entregue normalmente: `output("sw0-port2")`.

No segundo teste, o nível de detalhe será summary e neste modo teremos mais detalhes do processamento do sample packet incluindo qual pipeline que está sendo executado.

```

1 # ovn-trace --summary sw0 'inport == "sw0-port1" && eth.src == 00:00:00:00:00:01 && eth.dst == 00:00:00:00:00:02'
2 # reg14=0x1,vlan_tci=0x0000,d1_src=00:00:00:00:00:01,d1_dst=00:00:00:00:00:02,d1_type=0x0000
3 ingress(dp="sw0", inport="sw0-port1") {
4     next;
5     output = "sw0-port2";
6     output;
7     egress(dp="sw0", inport="sw0-port1", output="sw0-port2") {
8         output;

```

```

9      /* output to "sw0-port2", type "" */;
10    };
11  };

```

Com o output acima podemos determinar em mais detalhes o seguinte:

- o sample packet ingressa no Logical Switch sw0 vindo da porta sw0-port1 e é processado no pipeline ingress
- podemos verificar que o valor sw0-port2 está setado na variável output indicando que a porta de destino do sample packet é a porta sw0-port2
- o sample packet sai do pipeline ingress e entra no pipeline de egress e a variável output é setada para sw0-port2
- a action de output é executado no pipeline de egress que entrega o sample packet para sw0-port2

No terceiro teste, o nível de detalhe será detailed que é usado para debugs mais avançados:

```

1  # ovn-trace --detailed sw0 'inport == "sw0-port1" && eth.src == 00:00:00:00:00:01 && eth.dst == 00:00:00:00:00:02
2  # reg14=0x1,vlan_tci=0x0000,d1_src=00:00:00:00:00:01,d1_dst=00:00:00:00:00:02,d1_type=0x0000
3
4  ingress(dp="sw0", inport="sw0-port1")
5  -----
6  0. ls_in_port_sec_l2 (northd.c:5516): inport == "sw0-port1" && eth.src == {00:00:00:00:00:01}, priority 50, uuid
7  next;
8  24. ls_in_l2_lkup (northd.c:8299): eth.dst == 00:00:00:00:00:02, priority 50, uuid b467646a
9  output = "sw0-port2";
10 output;
11
12 egress(dp="sw0", inport="sw0-port1", output="sw0-port2")
13 -----
14 9. ls_out_port_sec_l2 (northd.c:5613): output == "sw0-port2" && eth.dst == {00:00:00:00:00:02}, priority 50, u
15 output;
16 /* output to "sw0-port2", type "" */

```

No output acima podemos visualizar o seguinte:

- o número das flow tables e nome dos estágios do pipeline que o sample packet será processado
- o flow match
- o priority do flow que está processando o sample packet
- uma referência do código fonte do OVN onde está sendo processado o sample packet

Com o ovn-trace também é possível determinar o motivo de um pacote ser dropped. Nesse exercício, anteriormente ativamos o **port security** e poderemos analisar o que ocorre quando um pacote chega na port sw0-port1 com um MAC Address de origem diferente do esperado:

```

1  # ovn-trace --detailed sw0 'inport == "sw0-port1" && eth.src == 00:00:00:00:00:ff && eth.dst == 00:00:00:00:00:02
2  # reg14=0x1,vlan_tci=0x0000,d1_src=00:00:00:00:00:ff,d1_dst=00:00:00:00:00:02,d1_type=0x0000
3
4  ingress(dp="sw0", inport="sw0-port1")
5  -----
6  0. ls_in_port_sec_l2: no match (implicit drop)

```

Acima o output nos mostra que o sample packet entrou no Logical Switch sw0 porém falhou em dar match nos flows estabelecidos na flow table 0, resultando no drop do pacote.

O nome **ls_in_port_sec_l2** da **flow table 0** significa **Logical Switch ingress L2 security policy**.

Também podemos exercitar o mesmo com o sample packet destinado para um MAC Address de destino inexistente:

```

1  # ovn-trace --detailed sw0 'inport == "sw0-port1" && eth.src == 00:00:00:00:00:01 && eth.dst == 00:00:00:00:00:ff
2  # reg14=0x1,vlan_tci=0x0000,d1_src=00:00:00:00:00:01,d1_dst=00:00:00:00:00:ff,d1_type=0x0000
3

```

```

4 ingress(dp="sw0", inport="sw0-port1")
5 -----
6 0. ls_in_port_sec_l2 (northd.c:5516): inport == "sw0-port1" && eth.src == {00:00:00:00:00:01}, priority 50, uui
7 next;
8 24. ls_in_l2_lkup (northd.c:7577): 1, priority 0, uuid 74f676ea
9 output = get_fdb(eth.dst);
10 next;
11 25. ls_in_l2_unknown (northd.c:7585): output == "none", priority 50, uuid 3f91009a
12 drop;

```

Verificando o output acima podemos determinar:

- o processamento do sample packet passou pela **flow table 0**
- entrou na **flow table 24** para executar o processo de **flow lookup** e identificar a porta de output (o pacote não tem porta de output)
- o pacote acabou sendo direcionado para a **flow table 25** que possui um flow que verifica se o pacote tem uma porta de output (none no nosso caso) e o mesmo foi dropped.

Para remover os Logical Flows criados anteriormente, execute os seguintes passos:

```

1 # ovn-nbctl lsp-del sw0-port1
2 # ovn-nbctl lsp-del sw0-port2
3 # ovn-nbctl ls-del sw0

```

Faça o double-check para analisar se todos os Logical Flows foram removidos:

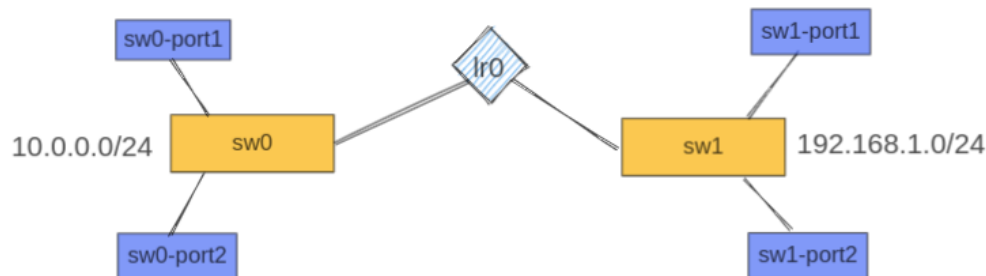
```

1 # ovn-nbctl show

```

Explorando os Logical Flows - cenário 2 [↗](#)

Diferentemente do exercício anterior que abordou os Logical Flows em um domínio L2, neste exercício iremos explorar os Logical Flows em uma topologia com várias subnets e um roteador conforme abaixo:



Siga os passos abaixo para realizar a criação dos Logical Flows conforme a topologia de rede acima:

```

1 # ovn-nbctl ls-add sw0
2 # ovn-nbctl lsp-add sw0 sw0-port1
3 # ovn-nbctl lsp-set-addresses sw0-port1 "00:00:00:00:00:01 10.0.0.51"
4 # ovn-nbctl lsp-set-port-security sw0-port1 "00:00:00:00:00:01 10.0.0.51"
5 # ovn-nbctl lsp-add sw0 sw0-port2
6 # ovn-nbctl lsp-set-addresses sw0-port2 "00:00:00:00:00:02 10.0.0.52"
7 # ovn-nbctl lsp-set-port-security sw0-port2 "00:00:00:00:00:02 10.0.0.52"
8
9 # ovn-nbctl ls-add sw1
10 # ovn-nbctl lsp-add sw1 sw1-port1
11 # ovn-nbctl lsp-set-addresses sw1-port1 "00:00:00:00:00:03 192.168.1.51"
12 # ovn-nbctl lsp-set-port-security sw1-port1 "00:00:00:00:00:03 192.168.1.51"
13 # ovn-nbctl lsp-add sw1 sw1-port2
14 # ovn-nbctl lsp-set-addresses sw1-port2 "00:00:00:00:00:04 192.168.1.52"

```

```

15 # ovn-nbctl lsp-set-port-security sw1-port2 "00:00:00:00:00:04 192.168.1.52"
16
17 # ovn-nbctl lr-add lr0
18 # ovn-nbctl lrp-add lr0 lrp0 00:00:00:00:ff:01 10.0.0.1/24
19 # ovn-nbctl lsp-add sw0 sw0-lrp0 \
20     -- set Logical_Switch_Port sw0-lrp0 type=router \
21     options:router-port=lrp0 addresses='"00:00:00:00:ff:01"'
22 # ovn-nbctl lrp-add lr0 lrp1 00:00:00:00:ff:02 192.168.1.1/24
23 # ovn-nbctl lsp-add sw1 sw1-lrp1 \
24     -- set Logical_Switch_Port sw1-lrp1 type=router \
25     options:router-port=lrp1 addresses='"00:00:00:00:ff:02"'

```

Verifique os Logical Flows criados:

```

1 # ovn-nbctl show
2 switch 380ccc2e-947f-4f9c-a33c-6bb0e823a660 (sw0)
3   port sw0-port2
4     addresses: ["00:00:00:00:00:02 10.0.0.52"]
5   port sw0-lrp0
6     type: router
7     addresses: ["00:00:00:00:ff:01"]
8     router-port: lrp0
9   port sw0-port1
10    addresses: ["00:00:00:00:00:01 10.0.0.51"]
11 switch 24e00387-f6be-4fea-9411-43f9b55e4f68 (sw1)
12   port sw1-port2
13     addresses: ["00:00:00:00:00:04 192.168.1.52"]
14   port sw1-lrp1
15     type: router
16     addresses: ["00:00:00:00:ff:02"]
17     router-port: lrp1
18   port sw1-port1
19     addresses: ["00:00:00:00:00:03 192.168.1.51"]
20 router 7bc34dd9-6e05-4823-85d5-5658f6fdf8ec (lr0)
21   port lrp1
22     mac: "00:00:00:00:ff:02"
23     networks: ["192.168.1.1/24"]
24   port lrp0
25     mac: "00:00:00:00:ff:01"
26     networks: ["10.0.0.1/24"]

```

Avalie todos os flows criados nos pipelines:

```

1 # ovn-sbctl lflow-list

```

Podemos avaliar com o ovn-trace a comunicação entre a porta sw0-port1 e sw1-port2 que irá requerer que o sample packet seja enviado através do Logical Router lr0:

```

1 # ovn-trace --detailed sw0 'inport == "sw0-port1" && eth.src == 00:00:00:00:00:01 && ip4.src == 10.0.0.51 && eth
2 # ip,reg14=0x1,vlan_tci=0x0000,d1_src=00:00:00:00:00:01,d1_dst=00:00:00:00:ff:01,nw_src=10.0.0.51,nw_dst=192.168
3
4 ingress(dp="sw0", inport="sw0-port1")
5 -----
6 0. ls_in_port_sec_l2 (northd.c:5516): inport == "sw0-port1" && eth.src == {00:00:00:00:00:01}, priority 50, uui
7   next;
8 1. ls_in_port_sec_ip (northd.c:5149): inport == "sw0-port1" && eth.src == 00:00:00:00:00:01 && ip4.src == {10.0
9   next;
10 24. ls_in_l2_lkup (northd.c:8299): eth.dst == 00:00:00:00:ff:01, priority 50, uuid c3661fee
11   output = "sw0-lrp0";

```

```

12     output;
13
14 egress(dp="sw0", inport="sw0-port1", output="sw0-lrp0")
15 -----
16 0. ls_out_pre_lb (northd.c:5666): ip && outport == "sw0-lrp0", priority 110, uuid c1352afe
17     next;
18 9. ls_out_port_sec_l2 (northd.c:5613): outport == "sw0-lrp0", priority 50, uuid e618ff95
19     output;
20     /* output to "sw0-lrp0", type "patch" */
21
22 ingress(dp="lr0", inport="lrp0")
23 -----
24 0. lr_in_admission (northd.c:10601): eth.dst == 00:00:00:00:ff:01 && inport == "lrp0", priority 50, uuid 67c9e3
25     xreg0[0..47] = 00:00:00:00:ff:01;
26     next;
27 1. lr_in_lookup_neighbor (northd.c:10745): 1, priority 0, uuid a9261fde
28     reg9[2] = 1;
29     next;
30 2. lr_in_learn_neighbor (northd.c:10754): reg9[2] == 1, priority 100, uuid 28da7fca
31     next;
32 10. lr_in_ip_routing_pre (northd.c:11004): 1, priority 0, uuid 37c4c10e
33     reg7 = 0;
34     next;
35 11. lr_in_ip_routing (northd.c:9517): ip4.dst == 192.168.1.0/24, priority 74, uuid 360a4761
36     ip.ttl--;
37     reg8[0..15] = 0;
38     reg0 = ip4.dst;
39     reg1 = 192.168.1.1;
40     eth.src = 00:00:00:00:ff:02;
41     outport = "lrp1";
42     flags.loopback = 1;
43     next;
44 12. lr_in_ip_routing_ecmp (northd.c:11079): reg8[0..15] == 0, priority 150, uuid a60d8ac1
45     next;
46 13. lr_in_policy (northd.c:11212): 1, priority 0, uuid 6b46dedf
47     reg8[0..15] = 0;
48     next;
49 14. lr_in_policy_ecmp (northd.c:11214): reg8[0..15] == 0, priority 150, uuid 7e313c80
50     next;
51 15. lr_in_arp_resolve (northd.c:11415): outport == "lrp1" && reg0 == 192.168.1.52, priority 100, uuid 6bfaac84
52     eth.dst = 00:00:00:00:00:04;
53     next;
54 19. lr_in_arp_request (northd.c:11895): 1, priority 0, uuid 91cbe8b6
55     output;
56
57 egress(dp="lr0", inport="lrp0", output="lrp1")
58 -----
59 0. lr_out_chk_dnat_local (northd.c:13120): 1, priority 0, uuid d504245c
60     reg9[4] = 0;
61     next;
62 6. lr_out_delivery (northd.c:11942): outport == "lrp1", priority 100, uuid 60967e1a
63     output;
64     /* output to "lrp1", type "patch" */
65
66 ingress(dp="sw1", inport="sw1-lrp1")
67 -----
68 0. ls_in_port_sec_l2 (northd.c:5516): inport == "sw1-lrp1", priority 50, uuid dd1cade9
69     next;

```

```

70 6. ls_in_pre_lb (northd.c:5663): ip && inport == "sw1-lrp1", priority 110, uuid c46b92ae
71   next;
72 24. ls_in_l2_lkup (northd.c:8299): eth.dst == 00:00:00:00:00:04, priority 50, uuid 5eeec0d7
73   output = "sw1-port2";
74   output;
75
76 egress(dp="sw1", inport="sw1-lrp1", output="sw1-port2")
77 -----
78 8. ls_out_port_sec_ip (northd.c:5149): output == "sw1-port2" && eth.dst == 00:00:00:00:00:04 && ip4.dst == {25
79   next;
80 9. ls_out_port_sec_l2 (northd.c:5613): output == "sw1-port2" && eth.dst == {00:00:00:00:00:04}, priority 50, u
81   output;
82   /* output to "sw1-port2", type "" */

```

Para remover os Logical Flows criados anteriormente, execute os seguintes passos:

```

1  # ovn-nbctl lsp-del sw0-port1
2  # ovn-nbctl lsp-del sw0-port2
3  # ovn-nbctl lsp-del sw0-lrp0
4  # ovn-nbctl ls-del sw0
5  # ovn-nbctl lsp-del sw1-port1
6  # ovn-nbctl lsp-del sw1-port2
7  # ovn-nbctl ls-del sw1
8  # ovn-nbctl lrp-del lrp0
9  # ovn-nbctl lrp-del lrp1
10 # ovn-nbctl lr-del lr0

```

Faça o double-check para analisar se todos os Logical Flows foram removidos:

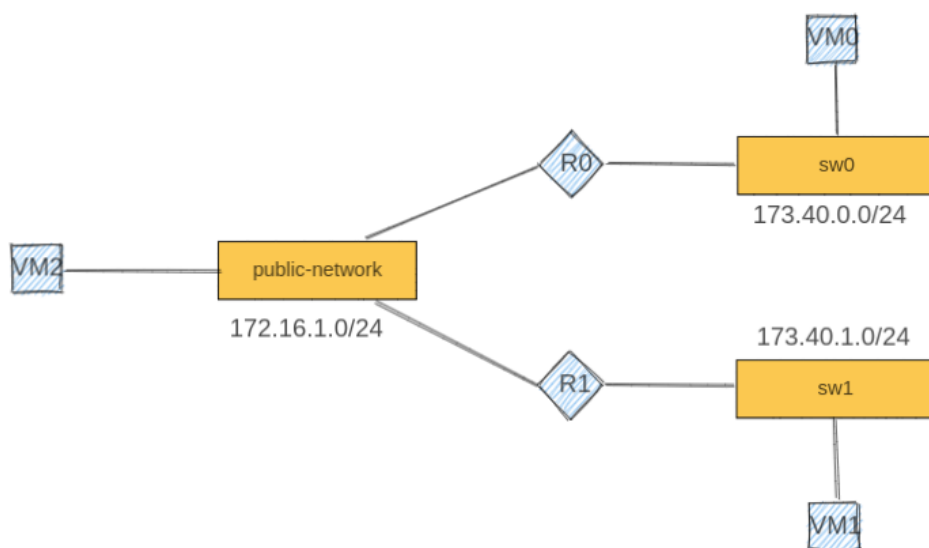
```

1  # ovn-nbctl show

```

Criando topologias de rede avançadas [↗](#)

Neste exercício, iremos explorar os Logical Flows da seguinte topologia de rede que abrange um cenário mais real criado pelo CMS(neutron):



Siga os passos abaixo para realizar a criação dos Logical Flows conforme a topologia de rede acima:

1 - Configurar o OVN Central e o ovn-controller na VM de sandbox

```

1 # ovn-nbctl set-connection ptcp:6641
2 # ovn-sbctl set-connection ptcp:6642
3 # ovs-vsctl set open . external_ids:system-id=HV1 external_ids:ovn-remote=tcp:127.0.0.1:6642 external_ids:ovn-enc
4 # systemctl restart ovn-controller

```

2 - Criar a bridge da rede pública (br-ext) e configurar esse Chassi como Chassi Gateway

```

1 # ovs-vsctl add-br br-ext
2 # ip link set br-ext up
3 # ovs-vsctl set Open_vSwitch . external-ids:ovn-bridge-mappings=provider:br-ext

```

3 - Configurar a rede pública que irá simular a rede de IP Público nesse cenário

```

1 # ovn-nbctl ls-add public-network
2 # ovn-nbctl lsp-add public-network ln_p1
3 # ovn-nbctl lsp-set-addresses ln_p1 unknown
4 # ovn-nbctl lsp-set-type ln_p1 localnet
5 # ovn-nbctl lsp-set-options ln_p1 network_name=provider

```

4 - Criar o router r0, a subnet sw0 e o namespace para a VM0. A VM0 terá o FIP 172.16.1.100 configurado

```

1 # ovn-nbctl lr-add r0
2 # ovn-nbctl lrp-add r0 r0_public 00:de:ad:ff:00:00 172.16.1.10/24
3 # ovn-nbctl lrp-add r0 r0_sw0 00:de:ad:fe:00:00 173.40.0.1/24
4 # ovn-nbctl lr-nat-add r0 dnat_and_snat 172.16.1.100 173.40.0.10
5 # ovn-nbctl lr-route-add r0 "0.0.0.0/0" 172.16.1.1
6 # ovn-nbctl lrp-set-gateway-chassis r0_public hv1
7
8 # ovn-nbctl ls-add sw0
9 # ovn-nbctl lsp-add sw0 sw0_r0
10 # ovn-nbctl lsp-set-type sw0_r0 router
11 # ovn-nbctl lsp-set-addresses sw0_r0 router
12 # ovn-nbctl lsp-set-options sw0_r0 router-port=r0_sw0
13
14 # ovn-nbctl lsp-add sw0 sw0-vm0-port1
15 # ovn-nbctl lsp-set-addresses sw0-vm0-port1 "00:de:ad:01:00:00 173.40.0.10"
16 # ovs-vsctl add-port br-int sw0-vm0-port1 -- set interface sw0-vm0-port1 type=internal external_ids:iface-id=sw0
17 # ovn-nbctl lsp-add public-network public_r0
18 # ovn-nbctl lsp-set-type public_r0 router
19 # ovn-nbctl lsp-set-addresses public_r0 router
20 # ovn-nbctl lsp-set-options public_r0 router-port=r0_public
21
22 # ip netns add vm0
23 # ip link set sw0-vm0-port1 netns vm0
24 # ip netns exec vm0 ip link set sw0-vm0-port1 address 00:de:ad:01:00:00
25 # ip netns exec vm0 ip addr add 173.40.0.10/24 dev sw0-vm0-port1
26 # ip netns exec vm0 ip link set sw0-vm0-port1 up
27 # ip netns exec vm0 ip route add default via 173.40.0.1

```

5 - Criar o router r1, a subnet sw1 e o namespace para a VM1. A VM1 terá o FIP 172.16.1.101 configurado

```

1 # ovn-nbctl lr-add r1
2 # ovn-nbctl lrp-add r1 r1_public 00:de:ad:ff:00:01 172.16.1.11/24
3 # ovn-nbctl lrp-add r1 r1_sw1 00:de:ad:fe:00:01 173.40.1.1/24
4 # ovn-nbctl lr-nat-add r1 dnat_and_snat 172.16.1.101 173.40.1.10
5 # ovn-nbctl lr-route-add r1 "0.0.0.0/0" 172.16.1.1
6 # ovn-nbctl lrp-set-gateway-chassis r1_public hv1
7
8 # ovn-nbctl ls-add sw1

```

```

9 # ovn-nbctl lsp-add sw1 sw1_r1
10 # ovn-nbctl lsp-set-type sw1_r1 router
11 # ovn-nbctl lsp-set-addresses sw1_r1 router
12 # ovn-nbctl lsp-set-options sw1_r1 router-port=r1_sw1
13
14 # ovn-nbctl lsp-add sw1 sw1-vm1-port1
15 # ovn-nbctl lsp-set-addresses sw1-vm1-port1 "00:de:ad:01:00:01 173.40.1.10"
16 # ovs-vsctl add-port br-int sw1-vm1-port1 -- set interface sw1-vm1-port1 type=internal external_ids:iface-id=sw1
17 # ovn-nbctl lsp-add public-network public_r1
18 # ovn-nbctl lsp-set-type public_r1 router
19 # ovn-nbctl lsp-set-addresses public_r1 router
20 # ovn-nbctl lsp-set-options public_r1 router-port=r1_public
21
22 # ip netns add vm1
23 # ip link set sw1-vm1-port1 netns vm1
24 # ip netns exec vm1 ip link set sw1-vm1-port1 address 00:de:ad:01:00:01
25 # ip netns exec vm1 ip addr add 173.40.1.10/24 dev sw1-vm1-port1
26 # ip netns exec vm1 ip link set sw1-vm1-port1 up
27 # ip netns exec vm1 ip route add default via 173.40.1.1

```

6 - Criar namespace da VM2 e plugar na rede pública. A VM0 terá o IP externo 172.16.1.200 configurado

```

1 # ip netns add vm2
2 # ovs-vsctl add-port br-ext vm2-port1 -- set interface vm2-port1 type=internal external_ids:iface-id=vm2-port1
3 # ip link set vm2-port1 netns vm2
4 # ip netns exec vm2 ip link set vm2-port1 address 00:00:00:00:00:01
5 # ip netns exec vm2 ip addr add 172.16.1.200/24 dev vm2-port1
6 # ip netns exec vm2 ip link set vm2-port1 up

```

Verifique a topologia de rede via Logical Flows:

```

1 # ovn-nbctl show
2 switch e54f75c0-40c8-4797-99aa-be4c3fed1640 (public-network)
3   port public_r1
4     type: router
5     router-port: r1_public
6   port ln_p1
7     type: localnet
8     addresses: ["unknown"]
9   port public_r0
10    type: router
11    router-port: r0_public
12 switch 767fc6b7-4366-46a2-9977-563024bb2655 (sw0)
13   port sw0_r0
14     type: router
15     router-port: r0_sw0
16   port sw0-vm0-port1
17     addresses: ["00:de:ad:01:00:00 173.40.0.10"]
18 switch ec100d9f-d0d8-45ed-96cb-0320bac2a231 (sw1)
19   port sw1_r1
20     type: router
21     router-port: r1_sw1
22   port sw1-vm1-port1
23     addresses: ["00:de:ad:01:00:01 173.40.1.10"]
24 router 54c35082-3308-4c34-b385-795119a7c7a5 (r0)
25   port r0_public
26     mac: "00:de:ad:ff:00:00"
27     networks: ["172.16.1.10/24"]
28   gateway chassis: [hv1]

```

```

29     port r0_sw0
30         mac: "00:de:ad:fe:00:00"
31         networks: ["173.40.0.1/24"]
32     nat 415f4c4a-1f8e-428d-939f-ae47a4212155
33         external ip: "172.16.1.100"
34         logical ip: "173.40.0.10"
35         type: "dnat_and_snat"
36 router a5519246-2de7-4151-82f7-c2bcfb79cb61 (r1)
37     port r1_sw1
38         mac: "00:de:ad:fe:00:01"
39         networks: ["173.40.1.1/24"]
40     port r1_public
41         mac: "00:de:ad:ff:00:01"
42         networks: ["172.16.1.11/24"]
43         gateway chassis: [hv1]
44     nat 1c9d8ce5-7cf1-4ab2-90eb-bf343f78b8ed
45         external ip: "172.16.1.101"
46         logical ip: "173.40.1.10"
47         type: "dnat_and_snat"

```

A VM2 está plugada na bridge br-ext via OVS para simular um host externo:

```

1 # ovs-vsctl show
2 9cba509a-d64f-488c-b6d8-462966aea0d1
3     Bridge br-ext
4         Port vm2-port1
5             Interface vm2-port1
6                 type: internal
7         Port patch-ln_p1-to-br-int
8             Interface patch-ln_p1-to-br-int
9                 type: patch
10                options: {peer=patch-br-int-to-ln_p1}
11     Port br-ext
12         Interface br-ext
13             type: internal

```

Vamos realizar a validação de conectividade de rede entre a VM0 e VM2:

```

1 # ip netns exec vm0 ping -c 4 172.16.1.200
2 PING 172.16.1.200 (172.16.1.200) 56(84) bytes of data.
3 64 bytes from 172.16.1.200: icmp_seq=1 ttl=63 time=0.062 ms
4 64 bytes from 172.16.1.200: icmp_seq=2 ttl=63 time=0.083 ms
5 64 bytes from 172.16.1.200: icmp_seq=3 ttl=63 time=0.078 ms
6 64 bytes from 172.16.1.200: icmp_seq=4 ttl=63 time=0.086 ms

```

Faça o mesmo teste entre a VM1 e VM2:

```

1 # ip netns exec vm1 ping -c 4 172.16.1.200
2 PING 172.16.1.200 (172.16.1.200) 56(84) bytes of data.
3 64 bytes from 172.16.1.200: icmp_seq=1 ttl=63 time=0.743 ms
4 64 bytes from 172.16.1.200: icmp_seq=2 ttl=63 time=0.469 ms
5 64 bytes from 172.16.1.200: icmp_seq=3 ttl=63 time=0.083 ms
6 64 bytes from 172.16.1.200: icmp_seq=4 ttl=63 time=0.084 ms

```

Podemos verificar algumas configurações em cada router, verifique no router r0 sua tabela de roteamento e as regras de NAT criadas:

```

1 # ovn-nbctl show r0
2 router 54c35082-3308-4c34-b385-795119a7c7a5 (r0)
3     port r0_public

```

```

4      mac: "00:de:ad:ff:00:00"
5      networks: ["172.16.1.10/24"]
6      gateway chassis: [hv1]
7      port r0_sw0
8      mac: "00:de:ad:fe:00:00"
9      networks: ["173.40.0.1/24"]
10     nat 415f4c4a-1f8e-428d-939f-ae47a4212155
11     external ip: "172.16.1.100"
12     logical ip: "173.40.0.10"
13     type: "dnat_and_snat"
14 # ovn-nbctl lr-route-list r0
15 IPv4 Routes
16 Route Table <main>:
17           0.0.0.0/0           172.16.1.1 dst-ip
18 # ovn-nbctl lr-nat-list r0
19 TYPE           EXTERNAL_IP       EXTERNAL_PORT   LOGICAL_IP       EXTERNAL_MAC     LOGICAL_PORT
20 dnat_and_snat   172.16.1.100      173.40.0.10

```

Conforme o output acima, o router r0 possui duas portas:

- r0_public - está conectada na rede pública, que possui o IP público 172.16.1.10
- r0_sw0 - está conectada subnet sw0 e com o IP 173.40.0.1 que é gateway interno

Sua tabela de roteamento possui uma rota default e uma regra de DNAT/SNAT do IP público 172.16.1.100 para o IP interno 173.40.0.10(VMO) que nesse caso é um NAT 1:1, ou seja, um FIP associado diretamente.

Fazendo um paralelo entre Logical Flows, Openflow e Physical flows [↗](#)

Este exercício é continuidade do anterior onde será feito um paralelo entre os tipos de flows que estudamos anteriormente.

Do lado do **OVN Central** sempre veremos os **Logical Flows**, porém o **ovn-controller** que executa nos Chassis fará a tradução para os flows **Openflow** e dessa forma o **ovs-vswitchd** irá programar o Datapath. No Datapath, será visto apenas os **Physical flows** que estão relacionados com o tráfego de rede que esteja sendo encaminhado no fast path.

Podemos iniciar a partir do OVN Central checando os Logical Flows implementados para o Logical Switch sw0 e para o Logical Router r0:

```

1 # ovn-sbctl lflow-list sw0
2 # ovn-sbctl lflow-list r0

```

Nos outputs, podemos determinar que o Logical Switch sw0 possui flows relacionados a camada de L2 e que o Logical Router r0 possui flows relacionados a L3 como roteamento e NAT.

No nível do Chassi, iremos observar os flows Openflow instalados conforme o que o ovn-controller programou. Usaremos o IP 173.40.0.10 para avaliar o output:

```

1 # ovs-ofctl dump-flows br-int| grep 173.40.0.10
2 cookie=0xd6b1181e, duration=8404.434s, table=14, n_packets=4, n_bytes=392, idle_age=7541, priority=100,ip,reg14=
3 cookie=0x2a651db1, duration=8300.511s, table=26, n_packets=0, n_bytes=0, idle_age=8300, priority=100,arp,reg14=0
4 cookie=0x8df098d4, duration=8393.630s, table=26, n_packets=0, n_bytes=0, idle_age=8393, priority=50,arp,metadata
5 cookie=0x2f2f6c2d, duration=8404.435s, table=41, n_packets=1149, n_bytes=112602, idle_age=0, priority=100,ip,reg
6 cookie=0xe664ee91, duration=8404.435s, table=43, n_packets=2, n_bytes=196, idle_age=8171, priority=162,ip,reg9=0
7 cookie=0x14353a34, duration=8404.435s, table=43, n_packets=7, n_bytes=686, idle_age=1146, priority=161,ip,reg15=

```

Para analisar os Physical Flows, primeiramente em outro terminal da sandbox VM, inicie um ping contínuo a partir da VM0 para a VM2:

```

1 # ip netns exec vm0 ping 172.16.1.200

```

Dessa forma, os Physical flows serão instalados no Datapath:

```

1 # ovs-dpctl dump-flows | grep 173.40.0.10
2 recirc_id(0x3d),in_port(5),eth(src=00:00:00:00:00:01,dst=00:de:ad:ff:00:00),eth_type(0x0800),ipv4(dst=173.40.0.10
3 recirc_id(0),in_port(3),eth(src=00:de:ad:01:00:00,dst=00:de:ad:fe:00:00),eth_type(0x0800),ipv4(src=173.40.0.10,ds
4 recirc_id(0x3b),in_port(3),eth(),eth_type(0x0800),ipv4(src=173.40.0.10,frag=no), packets:0, bytes:0, used:never,

```

Explorando os logs

Neste exercício, vamos explorar os logs do OVN Central no nível da SDN Controller e também no nível do Chassi.

Iniciando pelos arquivos de log do OVN Central:

- **OVN Northbound database** - `/var/log/ovn/ovsdb-server-nb.log`
- **ovn-northd** - `/var/log/ovn/ovn-northd.log`
- **OVN Southbound database** - `/var/log/ovn/ovsdb-server-sb.log`

Arquivos de log do Chassi:

- **ovn-controller** - `/var/log/ovn/ovn-controller.log`
- **ovsdb-server** - `/var/log/openvswitch/ovsdb-server.log`
- **ovs-vswitchd** - `/var/log/openvswitch/ovs-vswitchd.log`

Muitos logs são de INFO, alguns comuns por exemplo são das portas instanciadas no Chassi:

```

1 # tail -f /var/log/ovn/ovn-controller.log
2 2022-06-08T12:12:20.188Z|00024|binding|INFO|sw0-vm0-port1: Claiming 00:de:ad:01:00:00 173.40.0.10
3 2022-06-08T12:12:20.194Z|00025|binding|INFO|Setting lport sw0-vm0-port1 ovn-installed in OVS
4 2022-06-08T12:12:20.194Z|00026|binding|INFO|Setting lport sw0-vm0-port1 up in Southbound
5 2022-06-08T12:12:46.825Z|00027|binding|INFO|Claiming lport cr-r1_public for this chassis.
6 2022-06-08T12:12:46.825Z|00028|binding|INFO|cr-r1_public: Claiming 00:de:ad:ff:00:01 172.16.1.11/24
7 2022-06-08T12:13:00.754Z|00029|binding|INFO|Claiming lport sw1-vm1-port1 for this chassis.
8 2022-06-08T12:13:00.754Z|00030|binding|INFO|sw1-vm1-port1: Claiming 00:de:ad:01:00:01 173.40.1.10
9 2022-06-08T12:13:00.759Z|00031|binding|INFO|Setting lport sw1-vm1-port1 ovn-installed in OVS
10 2022-06-08T12:13:00.759Z|00032|binding|INFO|Setting lport sw1-vm1-port1 up in Southbound

```

Conclusão

Neste treinamento abordamos na prática como funciona a solução de OVN e conseguimos entrar em mais detalhes dos componentes envolvidos.

Abaixo existe o passo a passo para realizar o cleanup na VM utilizada para os exercícios.

```

1 # apt remove -y openvswitch-switch ovn-common ovn-host ovn-central
2 # rm -rf /var/lib/openvswitch
3 # rm -rf /var/log/openvswitch
4 # rm -rf /var/lib/ovn
5 # rm -rf /etc/openvswitch
6 # rm -f /etc/default/ovn-central
7 # rm -f /etc/default/ovn-host
8 # rm -f /etc/default/openvswitch-switch
9 # ip netns delete vm0
10 # ip netns delete vm1
11 # ip netns delete vm2
12 # reboot

```